Christopher Detrick

Full Conversation With Claude:

- Claude's responses are highlighted in orange
- My messages are not highlighted
- Claude's code was sent to me in an attachment that it updated as the conversation went on. The full final code is at the bottom of this document

```
from dataclasses import dataclass
import random
from drafter import *

@dataclass
class Term:
    term: str
    definition: str

user_terms = []
quiz_question_choices = []

# helper function
def find_removal_options(terms: list[Term]) -> list[str]:
    removal_options = []
    count = 0
    for term in terms:
        count = count + 1
        removal_options.append(str(count))
    return removal_options

# helper function
def is_term_format_valid(term_input: str) -> bool:
    if " - " not in term_input:
        return False

    hyphen_count = 0
    for index, char in enumerate(term_input):
        if index != len(term_input) - 1:
            if char == '-' and term_input[index - 1] == " " and term_input[index + 1] == " ":
                hyphen_count += 1
    if hyphen_count >= 2:
        return False
    return True

# helper function
```

```
def shuffle_terms(terms: list[Term]) -> list[Term]:
    shuffled_terms = random.sample(terms, len(terms))
    return shuffled_terms

# helper function, called by create_question_choices()
def determine_answer_choice_odds(terms: list[Term]) -> float:
    # determines the odds that each term in the list has of being one of the four answer choices
in a question
    odds = 4 / len(terms)
    return odds


# helper function
def create_question_choices(terms: list[Term]):
    odds = determine_answer_choice_odds(terms)
    for index, term in enumerate(terms):
        single_question_choices = [term.term]
        while len(single_question_choices) < 4:
            for t in terms:
                if len(single_question_choices) < 4:
                    if random.random() < odds and t.term not in single_question_choices:
                        single_question_choices.append(t.term)
        #shuffle
        single_question_choices = random.sample(single_question_choices,
len(single_question_choices))
        quiz_question_choices.append(single_question_choices)


# helper function
def present_question(correct_term: Term, all_terms: list[Term],
                     term_number: int, answer_choices: list[str]) -> bool:

    user_guess = 0
    print("\nQ" + str(term_number) + ": " + correct_term.definition + "\n" +
          "\t1| " + answer_choices[0] + "\n" +
          "\t2| " + answer_choices[1] + "\n" +
          "\t3| " + answer_choices[2] + "\n" +
          "\t4| " + answer_choices[3])
    while user_guess not in ['1','2','3','4']:
        user_guess = input("Type a number 1-4 to guess an option!\n")
    user_guess = int(user_guess)
    term_guessed = answer_choices[user_guess - 1]
    if term_guessed == correct_term.term:
        return True
```

```
    else:
        return False



def add_term():
    term_input = input("Enter a term and definition in the following format: Term - Definition\n")
    if is_term_format_valid(term_input) == False:
        print("Invalid format. No term has been added.")
        return
    term, definition = term_input.split(" - ")
    new_term = Term(term, definition)
    user_terms.append(new_term)
    print("New term added!\n\tTerm: " + term + "\n\tDefinition: " + definition)

def take_quiz():
    if len(user_terms) < 4:
        print("You must add at least 4 terms before you can take a quiz.")
        return

    quiz_question_choices.clear()
    shuffled_terms = shuffle_terms(user_terms)
    create_question_choices(shuffled_terms)
    score = 0

    for term_index, term in enumerate(shuffled_terms):
        is_q_answer_correct = present_question(term, shuffled_terms, term_index + 1,
                                               quiz_question_choices[term_index])
        if is_q_answer_correct == True:
            score += 1
            print("\nCorrect!")
        elif is_q_answer_correct == False:
            print("\nIncorrect!")
    print("Final Score: " + str(score) + "/" + str(len(shuffled_terms)))

def view_terms():
    if len(user_terms) == 0:
        print("There are no terms to view!")
        return
    count = 0
    for term in user_terms:
        count = count + 1
        print(str(count) + ": " + term.term + " - " + term.definition)
```

```
def remove_term():
    if len(user_terms) == 0:
        print("You need to add a term before you can remove one!")
        return
    removal_options = find_removal_options(user_terms)
    view_terms()
    user_input = input("Type a number to remove that term.\n")
    if user_input in removal_options:
        index_to_remove = int(user_input) - 1
        removed_term = user_terms[index_to_remove].term
        user_terms.pop(index_to_remove)
        print(removed_term + " successfully removed.")
    else:
        print("Invalid input. No term has been removed.")

def main():
    while True:
        user_input = input("\n- Type 'a' to add a term.\n- Type 'q' to take a multiple choice quiz." +
                           "\n- Type 'v' to view all terms.\n- Type 'r' to remove a term.\n")
        if user_input == 'a':
            add_term()
        if user_input == 'q':
            take_quiz()
        if user_input == 'v':
            view_terms()
        if user_input == 'r':
            remove_term()

main()
```

I am developing my final project for CS1. I have attached a table containing at least everything I want to be included. Here are the requirements for the project:

- At least 7 routes. At least 5 Pages, or 10 Fragments, or a combination thereof. The implementation of the State dataclass must have at least 4 fields. At least one field must be a list of class instances or a dictionary of class instances. This means that there must be a second dataclass. At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants. At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type. At least 3 meaningful if statements. At least 1 loop that iterates over an attribute of the list of

class instances or dictionary of class instances in a meaningful way. The site should have a legitimate purpose or functionality. No global variables (global constants are fine). You must use the Drafter library. Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

Requirement
Implementation
Purpose/Functionality

A 1-2 sentence description of the purpose or functionality of your site
My site will allow users to import vocabulary terms and definitions and take a shuffled multiple choice quiz to help them study.
State Dataclass Attributes

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.
user_terms (list[Term])
Stores all terms and definitions that the user has added via the “Add Term” or “Import Terms” pages.
quiz_terms (list[Term])
Stores a shuffled version of the user’s terms for the current quiz. The list will be shuffled when a new quiz begins, and questions answered incorrectly will be appended to the end of this list so that the user has to attempt them again.
current_question (int)
Stores the question number that the user is currently answering in a quiz. It will increase when the “Next Question” button is pressed.
score (int)
Stores the number of questions the user has answered correctly in the current quiz. It will increment when the user chooses the correct answer to a question.
feedback (str)
Stores the feedback displayed after the user answers a question. It will change to “Correct”

or “Incorrect” based on the user’s answer and will reset when a new question is shown.
Second Dataclass

Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)
Class name: Term
Attributes:
term (str)
Stores the name of the vocabulary word entered by the user. It will not be modified after it is created.
definition (str)
Stores the user-entered definition that goes with the term. It will not be modified after it is created.
Routes (minimum 7)

The names of your route functions
home
Displays the “Home” page
add_term
Displays the “Add Term” page
import_terms
Displays the “Import Terms” page
export_terms
Displays the “Export Terms” page
remove_term
Displays the “Remove Term” page
start_quiz
Starts a new quiz by shuffling the user terms and resetting the score and the current question
quiz
Displays the current quiz question and its four answer choices. Also processes the user’s answer and displays feedback
next_question
Moves to the next question after the user has answered. If the previous answer was

incorrect, it appends that question to the end of the quiz
Pages (minimum 5)

A brief description of the pages in your site
Home
The main page of the site that the user will return to. It shows a list of all the terms the user has added so far. This page features buttons that allow the user to add a term, import terms, export terms, remove a term, or take a multiple-choice quiz.
Add Term
This page features two textboxes that take in user input of a term and its definition. Once the user enters their input, they can click an “Add Term” button to add this term to the list of terms and return to the home page. The user can press “Cancel” at any time to return to the home page.
Import Terms
This page features one textbox that takes in terms and definitions in the form “Term - Definitions”. The user can add multiple terms at once, with each term separated by a new line. The main purpose of this page is to allow the user to paste a large amount of terms at once. Once the user enters into the textbox, they can press an “Import” button to officially add all of the terms, as long as they are in the right format. The user will be returned home after importing. The user can press “Cancel” at any time to return to the home page.
Export Terms
This page takes all of the terms that you have inputted and displays them in the form “Term - Definition”, with each term separated by a new line. The purpose of this page is to allow the user to save their inputted terms somewhere else on their computer so that they can import them back in the next time that they use the program. There is a “Home” button at the top so that the user can return to the home page at any time.
Remove Term
This page shows you all of the terms that you have added so far and has a button next to each of them to remove that term. After removing a term, the user returns to the home page. The user can also press “Cancel” to go back to the home page.
Quiz
This page has text at the top that displays a random definition out of all of the terms that the user has added. Below that, there are four buttons that each display a random term out of all of the terms the user has added. One of these terms will always be the correct term

that matches to the given definition. If the user presses the button that matches to the given definition, text will appear that says “Correct” and a button will appear that allows the user to go on to the next question. If the user presses one of the three incorrect buttons, text will appear that says “Incorrect,” it will tell the user the correct answer, and a button will appear that allows the user to go on to the next question. The question that was guessed incorrectly will be added to the end of the quiz so that the user has to attempt it again. When the user presses the button to go to the next question, the text at the top will change to a different definition and the answer choices will also change, resetting the cycle. At any point on this page, the user can press a “Quit” button in the top left to exit back to Home.
User Input (minimum 3)

Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes

For more information on these input types, see Drafter Documentation.
TextBox
Two are used on the “Add Term” page to allow the user to enter terms and definitions.
TextArea
Used on the “Import Terms” page to allow the user to paste multiple terms and definitions at once, each separated by a new line.
If Statements (minimum 3)
Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site
Check whether user_terms contains at least four terms before starting a quiz
If the user has less than four terms, a message will be shown saying that they need to add at least four terms before taking a quiz. Otherwise, the quiz will start.
Check whether the user’s imputed term and definition on the “Add Term” page is valid
If the term or definition are empty, an error message will be shown and no term will be added. Otherwise, a new instance of Term is created and added to the list.
Check if imported terms are in the required format
If an import does not contain the separator “ - “, it will not be added to the list. Otherwise, the terms will be added to the list.
Check whether the user’s chosen answer is equal to the correct answer
If the chosen answer is correct, the website shows “Correct” and increases the score by 1.

Otherwise, it will show “Incorrect” and reveal the correct answer.
Check whether the user has reached the end of the quiz
If there are no more questions remaining, the website will show the final score and an option to return to the Home page. Otherwise, the next question is shown.
Loop (minimum 1)
This loop must iterate over the list/dict of dataclass instances in your State attribute.
Description of this loop and why it is meaningful to the site
for term in state.user_terms
Iterates through every instance of Term in the user’s term list. This loop will be used on the “Home” page to display all of the terms and definitions. Similarly, it can also be used on the “Export Terms” page to combine all vocab terms into one string, and on the “Remove Term” page to show each term with a button to remove that term.

I have attached some code I have already written. The code I wrote was a prototype that I made, but I did not include all of the features and I also did not do any of the Drafter implementation. Can you finish the program, using Drafter instead of the terminal? Please refrain from rewriting parts of my code that do not need to be rewritten. Just change the parts the need to be changed for Drafter implementation and expand upon what I have. Make sure to meet all the requirements and finish it as I have envisioned. Also add proper docstrings to all the functions.

I finished the program in `vocab_quiz.py`, but I haven’t run it, because Drafter isn’t available in my environment. Please run it once and check the two Drafter features below.

What I kept from your prototype

- `Term`, `is_term_format_valid` and `shuffle_terms` are essentially unchanged.
- The `add_term`, `view_terms` and `remove_term` logic carries over, now split into Drafter pages.
- In `is_term_format_valid` I added `index > 0` to the loop check. Without it, `term_input[index - 1]` wraps around to the last character when `index` is 0.

What I changed or removed

- Globals. `user_terms` and `quiz_question_choices` now live in a `State` dataclass, since globals aren’t allowed. It has 7 fields: `user_terms`, `quiz_terms`, `current_question`, `score`, `feedback`, `choices` and `message`.

- Answer choices. `create_question_choices` now builds the four choices for one question at a time, with three random wrong terms plus the correct one. This is needed because missed questions get re-added mid-quiz. I removed `determine_answer_choice_odds` and `find_removal_options` because nothing uses them anymore.
- Quiz and input functions. `present_question`, `take_quiz`, `main` and the `input()`/`print()` code are replaced by Drafter routes and pages.

Routes and pages

- There are 12 routes: `index`, `home`, `add_term`, `save_term`, `import_terms`, `save_import`, `export_terms`, `remove_term`, `delete_term`, `start_quiz`, `quiz`, `next_question` and `results`.
- `index` only calls `home`. I added it because Drafter loads a route named `index` first.
- The form routes (`save_term`, `save_import`, `delete_term`) handle submissions, so each page route only displays.
- There are 7 pages (the 6 you listed plus the results page), 3 input fields (2 `TextBox`es and 1 `TextArea`), and loops over `state.user_terms` on the Home, Export and Remove pages.

Behavior you should know about

- Scoring. Since missed questions are re-added, scoring every correct answer would always end at the full score. So the score only counts correct answers on the first try. Retries are labeled “(retry)” on the question header.
- Duplicates. Duplicate term names are rejected, which also guarantees four distinct answer choices.
- Separator. Terms and definitions can’t contain `" - "`, because that would break import and export.
- Messages. Success and error messages are stored in `state.message` and shown once.

Drafter features to check

- Button arguments. I used `Button("Remove", delete_term, [Argument("index", index)])` to pass data to a route. If your course version of Drafter uses a different syntax, that line (and the matching one in `quiz`) is where to change it.

- Default parameter. `quiz(state, choice: str = "")` uses a default parameter so other routes can call `quiz(state)`. It's documented in the docstring. If Drafter complains, you could split the answer-checking into its own route.

The site runs well, but there are a few changes I would like to request. First of all, on most of the pages, all of the main features are on one line, which looks ugly and is annoying to navigate. For example, on the home page, all of the terms are listed within the same line, and the buttons are also on the same line after the terms. Change this so that the buttons are displayed at the top of the page, and the terms are each separated by a new line. Do something similar for every page. Also, for the remove page, put the remove button before the term and definition. Another feature I would like to add is the ability to see which words you got wrong and how many times you got it wrong in the results page. Another feature I would like to add is a "Remove All" button in the Remove terms page. Also, change the title on the home page to simply "Quiz Maker" I would also like to make the website look a bit nicer. If possible, colored text for certain elements such as green for correct and red for incorrect could look nice. Please update the code with these alterations.

I've updated `vocab_quiz.py` with all of your requested changes. I haven't run it, so please check each page.

Layout

- Home: the title comes first, then the buttons (Add, Import, Export, Remove, Take Quiz), then a divider, then each term on its own numbered line.
- Other pages: every page now has its navigation buttons at the top under the title (Cancel, Home, or Quit on the quiz), followed by a divider and the content.
- Form pages: the Add Term and Import submit buttons stay below their input boxes, since pressing them before filling in the form would be awkward.
- Separate lines: each item sits in a `Div`, which gives it its own line. The quiz answer buttons are now one per line too.

Features

- Remove page: each Remove button now comes before its term and definition. I also added a Remove All button, handled by a new `remove_all` route. It clears everything right away with no confirmation, so I can add a "Are you sure?" step if you'd like one.
- Missed terms on the results page: a new `missed` field in `State` (a dictionary of term name to number of wrong answers) is updated on every incorrect answer, including

retries. The results page loops over it and lists each missed term with a count like "missed 2 times". It resets when a new quiz starts, and if nothing was missed you get a green "Great job!" message.
- Title: the home page title is now "Quiz Maker", and I also set the browser tab title to match.

Colors

- Correct feedback and success messages are green, and incorrect feedback, error messages and missed terms are red.
- Headings are blue, and a perfect score is shown in green.
- The colors are global constants at the top of the file, so you can change them in one place.
- Error messages now start with "Error", which is how the code picks red over green.

Things to check

- I used Drafter's styling helpers (`bold`, `italic`, `change_color`, `HorizontalRule`, `set_website_title`) from memory, so if one raises an error in your version of Drafter, that line is the place to look.
- Imports that add zero terms now show in red as an error.

This looks great! One bug I found is that the import terms text area always seems to have two spaces at the start of it when opening the page. Could you fix this?

I made a one-line change in `vocab_quiz.py` that should remove the two leading spaces, but I haven't run it to confirm. The import text area was created with an empty string as its default text (`TextArea("import_text", "")`), and I suspect Drafter was padding that with whitespace when it built the box. It's now `TextArea("import_text")` with no default at all.

If the spaces are still there, they are harmless to your data. `save_import` already strips every line, so a leading space never ends up in a term. In that case, let me know and I can try another approach.

They are still there. This is not a major problem since it doesn't affect the usage of the website, but it irks me.

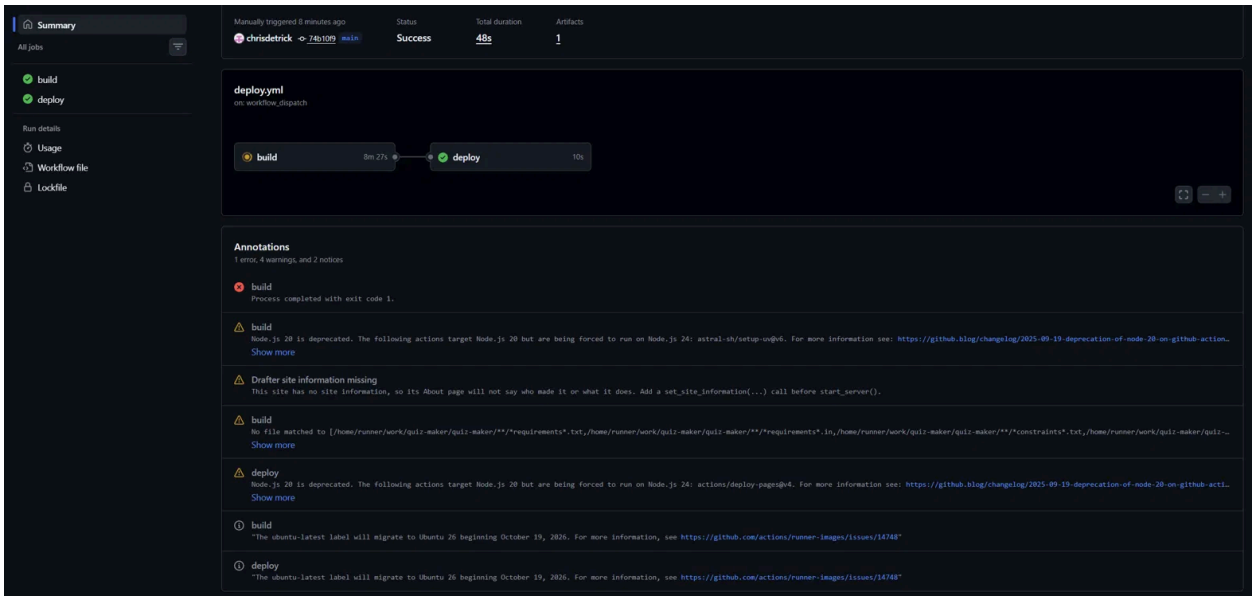
I've made another attempt in `vocab_quiz.py`, but I can't see how Drafter builds the page, so this is a best guess and I haven't run it.

My new theory is that the spaces come from the `Div` I wrapped around the text area. Drafter likely indents the HTML it generates, and for a text area that indentation ends up inside the box as visible spaces. The other text boxes aren't affected because they have no content between their tags. I removed the `Div` and now add the `TextArea` directly to the page. The `LineBreak` after it still puts the Import button on its own line.

If the spaces are still there, the problem is inside Drafter's own text area code, and I'd leave it, since `save_import` strips every line and your data is never affected. If you'd rather keep chasing it, the next thing to try would be checking Drafter's `TextArea` definition for how it writes its default text.

That worked! Thank you for your help

You're welcome! Glad that fixed it. Good luck with your final project, and feel free to come back if you want to add anything else or need help before you submit.



I got this message when uploading it to GitHub

Most of this is harmless, and the run itself succeeded: the status is **Success** and both the build and deploy jobs have green checks. Here's what each annotation means.

The one worth a look: the red "Process completed with exit code 1" on the build job means some step in the build returned an error. The workflow still went on to deploy, so that step is probably set to continue on error. To find it, click the **build** job on the left and look for the step with the red X. If your site loads correctly at its GitHub Pages URL, it's likely fine, but if

it's missing something or acting strangely, paste that step's log here and I'll help you work out what's wrong.

Warning from your code: "Drafter site information missing" is Drafter saying the site's About page has nothing on it. Add a call like this right before `start_server(...)` at the bottom of the file:

python

```python
set_site_information(
    author="Your Name",
    description="A study tool that lets you import vocabulary terms and take a shuffled multiple-choice quiz.",
    sources="",
    planning="",
    links=""
)
```

I'm going from memory on those parameter names, so check them against the Drafter docs if it complains. If your course asks you to document outside help or sources, the `sources` field is where that goes.

Safe to ignore:

- The **Node.js 20 deprecation** warnings are GitHub telling the authors of `setup-uv` and `deploy-pages` to update their actions. They don't affect your site.
- The *"No file matched requirements.txt"** warning means the setup step looked for a dependency file to cache and didn't find one. It's harmless.
- The two **ubuntu-latest notices** are just information about a future runner change.

Adding this code added five errors in Drafter

That was probably my mistake. I wrote the call from memory and filled in `sources`, `planning` and `links` with empty strings. I believe Drafter expects those three to be lists, so empty strings may be what triggered the errors. I can't see your error messages, so this is a guess.

The safest fix is to pass only the two fields that are plain text. Replace the whole call with this, again right before `start_server(...)`:

python

```python
set_site_information(
    author="Your Name",
    description="A study tool that lets you import vocabulary terms and take a shuffled multiple-choice quiz."
)
```

If you want the other fields filled in, they would need to be lists, like this:

python

```python
set_site_information(
    author="Your Name",
    description="A study tool that lets you import vocabulary terms and take a shuffled multiple-choice quiz.",
    sources=["Any source you used"],
    planning=[],
    links=[]
)
```

If the errors persist after trying the shorter version, paste the first error message here and I can work out what Drafter wants. The error text will usually say which field it objects to.

what is the red build error?

I can't tell from the screenshot. It only shows the generic message "Process completed with exit code 1", which GitHub adds whenever any step in the job ends with an error. The actual cause is in the build job's log.

How to find it

1. Open the run and click **build** in the left sidebar.
2. Look for the step with the red X and click it to expand it.
3. Scroll to the bottom of that step's output. The real error is usually in the last 10-20 lines, often a Python traceback or a line containing "Error".

What I can tell from the screenshot

- The Drafter "site information missing" warning appears in the build annotations, so the build step is running your Python script. The error is probably something that script printed.
- The deploy job succeeded, which means the build still produced and uploaded the site files (the Artifacts count shows 1). The failing step may be set to continue on error, or the failure may have happened after the files were created.
- The build graph shows 8m 27s next to a yellow icon, while the run's total duration is 48s. That doesn't add up, so the page may have been showing a stale or partial view. Refresh it and check whether the build job now shows a green check.

If you paste the last lines of the failing step here, I can tell you what it means. Also check whether your site loads and works at its GitHub Pages link, since that tells you whether the error affects anything.

Here is the run type checker from git hub
Run set -o pipefail
main.py:125: error: Incompatible return value type (got "Component | int | float | Sequence[Component | str | int | float | bool]", expected "Header") [return-value]
main.py:281: error: Argument 1 to "append" of "list" has incompatible type "Div"; expected "Header" [arg-type]
main.py:282: error: Argument 1 to "append" of "list" has incompatible type "HorizontalRule"; expected "Header" [arg-type]
main.py:283: error: Argument 1 to "append" of "list" has incompatible type "Div"; expected "Header" [arg-type]
main.py:286: error: Argument 1 to "append" of "list" has incompatible type "Div"; expected "Header" [arg-type]
main.py:287: error: Argument 1 to "append" of "list" has incompatible type "HorizontalRule"; expected "Header" [arg-type]
main.py:294: error: Argument 1 to "append" of "list" has incompatible type "Div"; expected "Header" [arg-type]
main.py:294: error: Argument 3 to "Button" has incompatible type "list[Argument]"; expected "Arguable | list[Arguable] | list[tuple[str, str | int | float | bool | list[Any] | dict[Any, Any] | None]] | dict[str, str | int | float | bool | list[Any] | dict[Any, Any] | None] | None" [arg-type]
main.py:294: note: "list" is invariant -- see

https://mypy.readthedocs.io/en/stable/common_issues.html#variance
main.py:294: note: Consider using "Sequence" instead, which is covariant
main.py:370: error: Argument 1 to "append" of "list" has incompatible type "Header"; expected "Div" [arg-type]
main.py:372: error: Argument 1 to "append" of "list" has incompatible type "Header"; expected "Div" [arg-type]
main.py:373: error: Argument 1 to "append" of "list" has incompatible type "HorizontalRule"; expected "Div" [arg-type]
main.py:375: error: Argument 1 to "append" of "list" has incompatible type "LineBreak"; expected "Div" [arg-type]
main.py:380: error: Argument 3 to "Button" has incompatible type "list[Argument]"; expected "Arguable | list[Arguable] | list[tuple[str, str | int | float | bool | list[Any] | dict[Any, Any] | None]] | dict[str, str | int | float | bool | list[Any] | dict[Any, Any] | None] | None" [arg-type]
main.py:380: note: "list" is invariant -- see https://mypy.readthedocs.io/en/stable/common_issues.html#variance
main.py:380: note: Consider using "Sequence" instead, which is covariant
main.py:386: error: Argument 1 to "append" of "list" has incompatible type "LineBreak"; expected "Div" [arg-type]
Found 14 errors in 1 file (checked 1 source file)
Error: Process completed with exit code 1.

That log is mypy, a static type checker your GitHub workflow runs on the code. It explains the red error: none of these are runtime bugs, and the site works. mypy is complaining about how I annotated a few things, and I should have caught these. There are three causes.

1. `make_title` has the wrong return type (the error on the `make_title` function)

I annotated it as `-> Header`, but `change_color(...)` returns a more general type. Change the import and the function signature:

python

```python
from typing import Any
```

(put this near the top with your other imports), then:

python

Python

```
def make_title(text: str, level: int = 1) -> Any:
```

2. Lists whose first item is a `Header` or `Div` (the `append` errors in `remove_term` and `quiz`)

When a list starts with just one item, mypy decides the whole list can only hold that type, so appending anything else fails. Annotate those two lists explicitly.

In `remove_term`:

python

Python

```
content: list[Any] = [make_title("Remove Term", 1)]
```

In `quiz`:

python

Python

```
content: list[Any] = [Div(Button("Quit", home))]
```

3. The `Button` argument lists (the two `Argument 3 to "Button"` errors)

mypy treats a `list[Argument]` as incompatible with what `Button` declares it accepts. The declared types include a dictionary, so switch to that form.

In `remove_term`, change the Remove button to:

python

Python

```
Button("Remove", delete_term, {"index": index})
```

In `quiz`, change the answer button to:

python

```
Button(option, quiz, {"choice": option})
```

You can also update the comment in `remove_term` to say the dictionary `{"index": index}` tells Drafter to give `delete_term` a parameter named `index`.

After making these changes, the build step should turn green. The dictionary form should behave the same way at runtime, but I haven't run it, so click a Remove button and an answer button on the live site to confirm both still work. If mypy reports any new errors after this, paste them and I'll sort them out.

can you send the entire code again with these changes

Ran a command, shared files

I've updated `vocab_quiz.py` with all the mypy fixes. It parses without syntax errors, but I couldn't run mypy or Drafter here, so I haven't confirmed the type errors are gone.

The changes are:

- `from typing import Any` is added to the imports.
- `make_title` now returns `Any` instead of `Header`.
- The `content` lists in `remove_term` and `quiz` are annotated as `list[Any]`.
- The Remove button now uses `{"index": index}`, and the answer buttons use `{"choice": option}`, instead of `Argument(...)` lists.

Two things to check before you copy it into your repo:

- This file doesn't include the `set_site_information(...)` call you tried earlier, so if you want it, add your working version back right before `start_server(...)`.

- The file is named `vocab_quiz.py`, but your workflow's errors refer to `main.py`. Paste the contents into `main.py` rather than adding a second file.

Once it's pushed, test a Remove button and a quiz answer button on the live site to confirm the dictionary form works. If mypy still reports errors, paste the new log and I'll sort out what's left.

can you update the code to meet these requirements?
Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of Chapter 1B1)
- Unit tested your functions where possible, being valid and thorough: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good docstrings as the first line of your function's body
- Decomposed complicated functions with helper functions

Also add this above start server:

```
set_site_information(
    author="Christopher Detrick",
    description="A study tool that lets you import vocabulary terms and take a shuffled multiple-choice quiz.",
    sources=["Claude"],
    planning=[],
    links=[]
)
```

I've updated `vocab_quiz.py` to cover the checklist and added your `set_site_information(...)` call right above `start_server`. I ran all the unit tests against a stand-in for Drafter, and they passed. I haven't run the file under real Drafter or mypy, so push it and check the build log.

Variable names

- I renamed vague names to descriptive ones: `t` is now `candidate_term`, `count` is `term_number` or `added_count`, `content` is `page_content`, and `index` is `term_index`.

- `choices` is now `answer_choices` and `missed` is now `missed_counts`.
- The route parameters changed to match the form and button names: the Add Term text boxes are `new_term` and `new_definition`, and the quiz button argument is `chosen_term`.

Helper functions

- `save_term`, `save_import`, `quiz` and `results` were broken up into small helpers: `find_term_error`, `parse_term_line`, `import_lines`, `make_import_message`, `build_export_text`, `record_missed_term`, `is_retry_question`, `process_answer` and `describe_times_missed`.
- The component-building code is also split out into `make_term_line`, `make_removal_row`, `make_choice_buttons`, `make_feedback_component` and `make_missed_content`.
- Magic values like `" - "`, `4` and the colors are now global constants.

Unit tests

- Every route has at least one test, and every logic helper is tested too.
- The routes `start_quiz` and `next_question` involve randomness, so their tests check properties instead of exact values. For example, they check that there are four choices and that the missed term is added to the end. Notes in the docstrings and the comment block above the tests say this.
- `shuffle_terms` and `create_question_choices` are random, and their docstrings say what the tests check. The component-building helpers can't be meaningfully compared, so the comment block lists them as checked by running the site.
- The route tests check the state inside the returned page, not the page layout.

Types, docstrings and line length

- Every function has type hints and a docstring as its first line, and no line is over 120 characters. A few lines run past 100, so shorten them if your instructor's 80-character rule of thumb matters to you.

Things to watch for

- I assumed `assert_equal` comes in with `from drafter import *`. If you get a "not defined" error, add `from bakery import assert_equal` to the imports.
- The tests also assume the returned page has a `.state` attribute.
- If mypy complains about anything new, paste the log and I'll fix it.

can you move the unit tests so that they are under the function they are testing. Also, add from bakery import assert_equal to the top

I updated `vocab_quiz.py` so each function is followed by its unit test, and added `from bakery import assert_equal` to the imports. I ran all the tests against a stand-in for Drafter and they passed, but I haven't run the file under real Drafter or mypy.

How the tests are laid out

- Each function is followed by a `test_...` function, for example `test_term_exists` under `term_exists` and `test_home` under `home`.
- The tests don't run right where they're written. Instead, a `run_all_tests()` function near the bottom calls them all just before `set_site_information(...)` and `start_server(...)`.
- I did this for two reasons:
    - The routes link to each other, so calling `home` immediately after defining it would fail because `add_term` and the others don't exist yet.
    - Keeping the test data inside functions avoids creating global variables, which your requirements don't allow.
- A comment at the top of the file and the docstring of `run_all_tests()` explain this.

Other changes

- The test setup helpers (`make_test_terms`, `make_test_state`, `make_quiz_state`) sit in a small section right after the `State` class so every test can use them.
- The functions that only build Drafter components (`make_title`, `make_term_line`, `make_removal_row`, `make_choice_buttons`, `make_feedback_component` and `make_missed_content`) have no tests. Their docstrings now say they can't be unit tested and are checked by running the site.
- Nothing else about the program's behavior changed.

If your instructor wants plain `assert_equal(...)` lines directly under each function instead of test functions, that can be done for the helper functions, but the route tests would need the shared `run_all_tests()` approach for the reasons above.

Don't put the assert equals inside of functions

I've moved the unit tests out of functions in `vocab_quiz.py`. Each function is now followed by plain `assert_equal(...)` lines at the top level of the file, and `run_all_tests()` is gone. I ran the file with a stand-in for Drafter and every test passed. I haven't run it under real Drafter or mypy, so please check the site and the build log.

To make that work, I had to change two things.

Buttons now use route names (the main thing to check). Routes link to each other, so a test directly under `home` would crash because `add_term` wasn't defined yet. To fix this, every `Button` now takes the route's name as a string, such as `Button("Add Term", "add_term")`, instead of the function itself. I'm fairly confident Drafter accepts this, but please click through every button on the site once to make sure. I documented this in a comment near the top, since your requirements ask for any techniques beyond the course to be explained. I also moved the routes around so each one only calls routes defined above it.

No global variables. Top-level test code can't use variables without creating globals, so each test is a single `assert_equal(...)` line. Where a test needed to change a state first, I added a small test helper next to the function, such as `state_after_answer`, `terms_after_import` and `missed_counts_after_records`. Each helper has a docstring, and a scan found no top-level variables besides the constants.

Everything else is the same as before. The component-building functions still have docstrings saying they can't be unit tested, and the random functions are still tested by their properties.

Claude's Final Code:

```
from dataclasses import dataclass
import random
from typing import Any, Optional
from drafter import *
from bakery import assert_equal

# Global constants (allowed)
CORRECT_FEEDBACK = "Correct!"   # also compared against state.feedback to see if the last answer was right
TITLE_COLOR = "#2c5aa0"        # blue used for page headings
GOOD_COLOR = "green"           # correct answers and success messages
BAD_COLOR = "red"              # incorrect answers and error messages
TERM_SEPARATOR = " - "         # separates a term from its definition when importing/exporting
MIN_TERMS_FOR_QUIZ = 4         # one correct answer plus three wrong choices
NUMBER_OF_CHOICES = 4          # answer buttons shown per question

set_website_title("Quiz Maker")

# Advanced technique (not covered in the course): every Button below is given the NAME of its
# route as a string (for example Button("Home", "home")) instead of the route function itself.
```

```
# This lets a button point to a route that is defined LATER in the file, so that each route can be
# unit tested directly underneath it. Drafter treats a route's name and the route function the
same way.


@dataclass
class Term:
    """A single vocabulary term and its definition. Neither field changes after creation."""
    term: str
    definition: str


@dataclass
class State:
    """
    Holds everything the website needs to remember between pages.

    user_terms: every Term the user has added or imported.
    quiz_terms: the shuffled Terms for the current quiz (missed Terms are appended to the end).
    current_question: index in quiz_terms of the question being answered.
    score: number of questions answered correctly on the first try in the current quiz.
    feedback: text shown after answering ("" means the question is not answered yet).
    answer_choices: the four answer choices (term names) for the current question.
    message: a one-time status or error message shown on the next page that displays it.
    missed_counts: maps a term's name to how many times it was answered incorrectly in the
current quiz.
    """
    user_terms: list[Term]
    quiz_terms: list[Term]
    current_question: int
    score: int
    feedback: str
    answer_choices: list[str]
    message: str
    missed_counts: dict[str, int]


# ------------------------------------------------------------------------------------
# Helpers used only by the unit tests
# (The tests below are plain assert_equal lines with no variables, so a few small helpers
# build the Terms and States they need.)
# ------------------------------------------------------------------------------------

def make_test_terms() -> list[Term]:
```

```
    """Return a fresh list of four Terms to use in the unit tests."""
    return [Term("cat", "a small pet"), Term("dog", "a loyal pet"),
            Term("fish", "lives in water"), Term("bird", "can fly")]


def make_test_state(terms: list[Term]) -> State:
    """Return a fresh State holding the given terms, with every other field at its starting value."""
    return State(terms, [], 0, 0, "", [], "", {})


def make_quiz_state(current_question: int, quiz_terms: list[Term], feedback: str) -> State:
    """Return a State in the middle of a quiz over the four test terms, for the quiz unit tests."""
    return State(make_test_terms(), quiz_terms, current_question, 0, feedback,
                 ["cat", "dog", "fish", "bird"], "", {})


# ------------------------------------------------------------------------------
# Helper functions (logic)
# ------------------------------------------------------------------------------

def term_exists(terms: list[Term], name: str) -> bool:
    """Return True if a Term with this name (ignoring upper/lower case) is already in terms."""
    for existing_term in terms:
        if existing_term.term.lower() == name.lower():
            return True
    return False


assert_equal(term_exists(make_test_terms(), "cat"), True)
assert_equal(term_exists(make_test_terms(), "CAT"), True)
assert_equal(term_exists(make_test_terms(), "horse"), False)
assert_equal(term_exists([], "cat"), False)


def is_term_format_valid(term_input: str) -> bool:
    """
    Return True if term_input looks like 'Term - Definition'.

    The text must contain ' - ' exactly once, so that it can be split into a term and a definition.
    """
    if TERM_SEPARATOR not in term_input:
        return False

    hyphen_count = 0
```

```
    for index, char in enumerate(term_input):
        # (index > 0 added so that index - 1 never wraps around to the end of the string)
        if index > 0 and index != len(term_input) - 1:
            if char == '-' and term_input[index - 1] == " " and term_input[index + 1] == " ":
                hyphen_count += 1
    if hyphen_count >= 2:
        return False
    return True


assert_equal(is_term_format_valid("cat - a small pet"), True)
assert_equal(is_term_format_valid("well-known - famous"), True)
assert_equal(is_term_format_valid("no separator here"), False)
assert_equal(is_term_format_valid("a - b - c"), False)
assert_equal(is_term_format_valid(""), False)


def shuffle_terms(terms: list[Term]) -> list[Term]:
    """
    Return a new, randomly ordered list containing the same Terms (the original is unchanged).

    The exact order is random, so it can't be unit tested directly; the tests only check
    that the length and the Terms stay the same.
    """
    shuffled_terms = random.sample(terms, len(terms))
    return shuffled_terms


assert_equal(len(shuffle_terms(make_test_terms())), 4)
assert_equal(term_exists(shuffle_terms(make_test_terms()), "cat"), True)
assert_equal(term_exists(shuffle_terms(make_test_terms()), "bird"), True)
assert_equal(shuffle_terms([]), [])


def create_question_choices(correct_term: Term, terms: list[Term]) -> list[str]:
    """
    Return a shuffled list of four term names: the correct one plus three random wrong ones.

    The wrong choices are picked from the other Terms in `terms` (the caller must make sure
    there are at least three other Terms). Because the result is random, the tests only check
    its length, that it holds the correct term, and that it has no duplicates.
    """
    wrong_options = []
    for candidate_term in terms:
```

```
        if candidate_term.term != correct_term.term:
            wrong_options.append(candidate_term.term)
    answer_options = random.sample(wrong_options, NUMBER_OF_CHOICES - 1)
    answer_options.append(correct_term.term)
    # shuffle so the correct answer isn't always in the same position
    return random.sample(answer_options, len(answer_options))


assert_equal(len(create_question_choices(make_test_terms()[0], make_test_terms())), 4)
assert_equal("cat" in create_question_choices(make_test_terms()[0], make_test_terms()), True)
assert_equal(len(set(create_question_choices(make_test_terms()[0], make_test_terms()))), 4)


def take_message(state: State) -> str:
    """Return the pending one-time message from the state and clear it so it only shows once."""
    pending_message = state.message
    state.message = ""
    return pending_message


def message_left_after_take(message: str) -> str:
    """Test helper: put message in a new State, call take_message on it, and return what is left."""
    message_state = make_test_state([])
    message_state.message = message
    take_message(message_state)
    return message_state.message


def taken_message(message: str) -> str:
    """Test helper: put message in a new State and return what take_message gives back."""
    message_state = make_test_state([])
    message_state.message = message
    return take_message(message_state)


assert_equal(taken_message("Hello"), "Hello")
assert_equal(taken_message(""), "")
assert_equal(message_left_after_take("Hello"), "")


def find_term_error(term_name: str, definition: str, terms: list[Term]) -> str:
    """
    Return an error message if this term/definition can't be added to terms, or "" if it is fine.
```

```
    A term can't be added if either part is empty, if either part contains ' - ' (that would
    break import/export), or if the term name already exists.
    """
    if term_name == "" or definition == "":
        return "Error: the term and the definition cannot be empty. No term has been added."
    # Reuses is_term_format_valid so a term/definition can't contain ' - '
    if not is_term_format_valid(term_name + TERM_SEPARATOR + definition):
        return "Error: the term and definition cannot contain ' - '. No term has been added."
    if term_exists(terms, term_name):
        return "Error: that term has already been added. No term has been added."
    return ""


assert_equal(find_term_error("horse", "an animal", make_test_terms()), "")
assert_equal(find_term_error("", "an animal", make_test_terms()).startswith("Error"), True)
assert_equal(find_term_error("horse", "", make_test_terms()).startswith("Error"), True)
assert_equal(find_term_error("a - b", "an animal", make_test_terms()).startswith("Error"), True)
assert_equal(find_term_error("cat", "another one", make_test_terms()).startswith("Error"), True)


def parse_term_line(line: str) -> Optional[Term]:
    """
    Turn one line of 'Term - Definition' text into a Term.

    Return None if the line is in the wrong format or if the term or definition is empty.
    """
    stripped_line = line.strip()
    if not is_term_format_valid(stripped_line):
        return None
    # the second argument (1) means "split at most once"
    term_name, definition = stripped_line.split(TERM_SEPARATOR, 1)
    term_name = term_name.strip()
    definition = definition.strip()
    if term_name == "" or definition == "":
        return None
    return Term(term_name, definition)


assert_equal(parse_term_line("cat - a small pet"), Term("cat", "a small pet"))
assert_equal(parse_term_line("  cat  -  a small pet  "), Term("cat", "a small pet"))
assert_equal(parse_term_line("not a term line"), None)
assert_equal(parse_term_line("cat - a - b"), None)
assert_equal(parse_term_line(" - missing term"), None)
```

```
def import_lines(import_text: str, terms: list[Term]) -> tuple[int, int]:
    """
    Add every valid, non-duplicate 'Term - Definition' line in import_text to terms.

    Blank lines are ignored. Return (number of terms added, number of lines skipped).
    """
    added_count = 0
    skipped_count = 0
    # splitlines() splits a string into a list of its lines (works for any kind of line ending)
    for line in import_text.splitlines():
        if line.strip() == "":
            continue
        parsed_term = parse_term_line(line)
        if parsed_term is None or term_exists(terms, parsed_term.term):
            skipped_count += 1
        else:
            terms.append(parsed_term)
            added_count += 1
    return added_count, skipped_count


def terms_after_import(import_text: str, terms: list[Term]) -> list[Term]:
    """Test helper: run import_lines on terms and return the list of terms afterwards."""
    import_lines(import_text, terms)
    return terms


assert_equal(import_lines("dog - a loyal pet\n\nbad line\ncat - duplicate\nfish - lives in water",
                          [Term("cat", "a small pet")]), (2, 2))
assert_equal(import_lines("", []), (0, 0))
assert_equal(terms_after_import("dog - a loyal pet\n\nbad line\ncat - duplicate\nfish - lives in
water",
                                [Term("cat", "a small pet")]),
             [Term("cat", "a small pet"), Term("dog", "a loyal pet"), Term("fish", "lives in water")])


def make_import_message(added_count: int, skipped_count: int) -> str:
    """Return the message shown after an import (starts with 'Error' if nothing was added)."""
    message = "Imported " + str(added_count) + " term(s). Skipped " + str(skipped_count) + "
line(s)."
    if added_count == 0:
        return "Error: " + message
    return message
```

```
assert_equal(make_import_message(2, 1), "Imported 2 term(s). Skipped 1 line(s).")
assert_equal(make_import_message(0, 3), "Error: Imported 0 term(s). Skipped 3 line(s).")


def build_export_text(terms: list[Term]) -> str:
    """Return all terms as 'Term - Definition' lines, one per line, ready to be imported again."""
    export_text = ""
    # Loop over the Term instances to combine them into one string
    for term in terms:
        export_text = export_text + term.term + TERM_SEPARATOR + term.definition + "\n"
    return export_text


assert_equal(build_export_text([]), "")
assert_equal(build_export_text([Term("cat", "a small pet"), Term("dog", "a loyal pet")]),
             "cat - a small pet\ndog - a loyal pet\n")


def record_missed_term(missed_counts: dict[str, int], term_name: str) -> None:
    """Add one to the number of times term_name was missed (starting at 1 the first time)."""
    if term_name in missed_counts:
        missed_counts[term_name] += 1
    else:
        missed_counts[term_name] = 1


def missed_counts_after_records(term_names: list[str]) -> dict[str, int]:
    """Test helper: call record_missed_term for each name in order and return the resulting
dictionary."""
    missed_counts: dict[str, int] = {}
    for term_name in term_names:
        record_missed_term(missed_counts, term_name)
    return missed_counts


assert_equal(missed_counts_after_records([]), {})
assert_equal(missed_counts_after_records(["cat"]), {"cat": 1})
assert_equal(missed_counts_after_records(["cat", "dog", "cat"]), {"cat": 2, "dog": 1})


def is_retry_question(state: State) -> bool:
    """
```

```
    Return True if the current question is a retry of a missed question.

    Retries are appended after the original terms, so their position is at or past len(user_terms).
    """
    return state.current_question >= len(state.user_terms)


assert_equal(is_retry_question(make_quiz_state(3, make_test_terms(), "")), False)
assert_equal(is_retry_question(make_quiz_state(4, make_test_terms(), "")), True)


def process_answer(state: State, chosen_term: str) -> None:
    """
    Check chosen_term against the current question and update feedback, score and
missed_counts.

    Does nothing if no term was chosen or the question was already answered. Only first-pass
    questions add to the score; every incorrect answer (including retries) is recorded as missed.
    """
    if chosen_term == "" or state.feedback != "":
        return
    correct_term = state.quiz_terms[state.current_question]
    if chosen_term == correct_term.term:
        state.feedback = CORRECT_FEEDBACK
        if not is_retry_question(state):
            state.score += 1
    else:
        state.feedback = "Incorrect! The correct answer was: " + correct_term.term
        record_missed_term(state.missed_counts, correct_term.term)


def state_after_answer(current_question: int, quiz_terms: list[Term], feedback: str,
                       chosen_term: str) -> State:
    """Test helper: build a quiz State, call process_answer on it, and return the State
afterwards."""
    answer_state = make_quiz_state(current_question, quiz_terms, feedback)
    process_answer(answer_state, chosen_term)
    return answer_state


assert_equal(state_after_answer(0, make_test_terms(), "", "cat").feedback,
CORRECT_FEEDBACK)
assert_equal(state_after_answer(0, make_test_terms(), "", "cat").score, 1)
assert_equal(state_after_answer(0, make_test_terms(), "", "dog").feedback,
```

```
            "Incorrect! The correct answer was: cat")
assert_equal(state_after_answer(0, make_test_terms(), "", "dog").score, 0)
assert_equal(state_after_answer(0, make_test_terms(), "", "dog").missed_counts, {"cat": 1})
# already answered: nothing changes
assert_equal(state_after_answer(0, make_test_terms(), CORRECT_FEEDBACK, "cat").score,
0)
# no choice made: nothing changes
assert_equal(state_after_answer(0, make_test_terms(), "", "").feedback, "")
# a retry (position 4 is past the four original terms) is correct but doesn't add to the score
assert_equal(state_after_answer(4, make_test_terms() + [Term("cat", "a small pet")], "",
"cat").feedback,
             CORRECT_FEEDBACK)
assert_equal(state_after_answer(4, make_test_terms() + [Term("cat", "a small pet")], "",
"cat").score, 0)


def describe_times_missed(times_missed: int) -> str:
    """Return text like ' - missed 1 time' or ' - missed 3 times'."""
    if times_missed == 1:
        return " - missed 1 time"
    return " - missed " + str(times_missed) + " times"


assert_equal(describe_times_missed(1), " - missed 1 time")
assert_equal(describe_times_missed(3), " - missed 3 times")


# ---------------------------------------------------------------------------
# Helper functions (page components)
# ---------------------------------------------------------------------------

def make_title(text: str, level: int = 1) -> Any:
    """
    Return a Header whose text is colored with the site's title color.

    Can't be unit tested: it only builds a styled Drafter component, so it is checked by running the
site.
    """
    return change_color(Header(text, level), TITLE_COLOR)


def make_message_content(message: str) -> list[Any]:
    """
    Turn a message into page content: red if it starts with 'Error', green otherwise.
```

```
    Returns an empty list if there is no message, so the result can be added to a page's content.
    (Only the length of the result is unit tested, since the styled components can't be compared.)
    """
    if message == "":
        return []
    if message.startswith("Error"):
        return [Div(bold(change_color(message, BAD_COLOR)))]
    return [Div(bold(change_color(message, GOOD_COLOR)))]


assert_equal(make_message_content(""), [])
assert_equal(len(make_message_content("Success")), 1)
assert_equal(len(make_message_content("Error: bad")), 1)


def make_term_line(term_number: int, term: Term) -> Any:
    """
    Return a Div showing '1. Term - Definition' (with the term in bold) on its own line.

    Can't be unit tested: it only builds Drafter components, so it is checked by running the site.
    """
    return Div(str(term_number) + ". ", bold(term.term), TERM_SEPARATOR + term.definition)


def make_removal_row(term_index: int, term: Term) -> Any:
    """
    Return a Div with a Remove button (placed before the term) for the Term at term_index.

    Can't be unit tested: it only builds Drafter components, so it is checked by running the site.
    """
    return Div(Button("Remove", "delete_term", {"term_index": term_index}),
               "  " + str(term_index + 1) + ". ", bold(term.term), TERM_SEPARATOR + term.definition)


def make_choice_buttons(answer_choices: list[str]) -> list[Any]:
    """
    Return one Div per answer choice, each holding a button that submits that choice.

    Can't be unit tested: it only builds Drafter components, so it is checked by running the site.
    """
    choice_buttons = []
    for answer_choice in answer_choices:
```

```
        choice_buttons.append(Div(Button(answer_choice, "quiz", {"chosen_term":
answer_choice})))
    return choice_buttons


def make_feedback_component(feedback: str) -> Any:
    """
    Return the feedback text in bold: green if it is the correct message, red otherwise.

    Can't be unit tested: it only builds styled Drafter components, so it is checked by running the
site.
    """
    if feedback == CORRECT_FEEDBACK:
        return Div(bold(change_color(feedback, GOOD_COLOR)))
    return Div(bold(change_color(feedback, BAD_COLOR)))


def make_missed_content(missed_counts: dict[str, int]) -> list[Any]:
    """
    Return one red line per missed term, or a green message if no terms were missed.

    Can't be unit tested: it only builds styled Drafter components, so it is checked by running the
site.
    """
    if len(missed_counts) == 0:
        return [Div(bold(change_color("You didn't miss any terms. Great job!", GOOD_COLOR)))]
    missed_content = []
    # Loop over the dictionary; .items() gives each key (term name) and its value (times missed)
    for term_name, times_missed in missed_counts.items():
        missed_content.append(Div(bold(change_color(term_name, BAD_COLOR)),
                                  describe_times_missed(times_missed)))
    return missed_content


# ---------------------------------------------------------------------------------
# Routes
# (The route tests check the state inside the returned Page. Routes call take_message, so a
# message set before a page is shown is cleared afterwards. Routes are ordered so that each
# route only calls routes that are defined above it.)
# ---------------------------------------------------------------------------------

@route
def home(state: State) -> Page:
    """Home page: buttons to every other page at the top, then each term on its own line."""
```

```
    page_content: list[Any] = [
        make_title("Quiz Maker", 1),
        Div(Button("Add Term", "add_term"),
            Button("Import Terms", "import_terms"),
            Button("Export Terms", "export_terms"),
            Button("Remove Term", "remove_term"),
            Button("Take Quiz", "start_quiz")),
        HorizontalRule()
    ]
    page_content.extend(make_message_content(take_message(state)))

    page_content.append(make_title("Your Terms (" + str(len(state.user_terms)) + ")", 2))
    if len(state.user_terms) == 0:
        page_content.append(Div(italic("You haven't added any terms yet.")))
    # Loop over the Term instances stored in the state; each Div is its own line
    term_number = 0
    for term in state.user_terms:
        term_number = term_number + 1
        page_content.append(make_term_line(term_number, term))
    return Page(state, page_content)


def make_home_test_state() -> State:
    """Test helper: return a State with four terms and a pending message, for testing home."""
    home_state = make_test_state(make_test_terms())
    home_state.message = "Some message"
    return home_state


assert_equal(home(make_home_test_state()).state, make_test_state(make_test_terms()))
assert_equal(home(make_test_state([])).state, make_test_state([]))


@route
def index(state: State) -> Page:
    """Default route Drafter loads first; it simply shows the home page."""
    return home(state)


assert_equal(index(make_test_state(make_test_terms())).state,
make_test_state(make_test_terms()))


@route
```

```
def add_term(state: State) -> Page:
    """Add Term page: two text boxes for the term and its definition."""
    page_content: list[Any] = [
        make_title("Add Term", 1),
        Div(Button("Cancel", "home")),
        HorizontalRule()
    ]
    page_content.extend(make_message_content(take_message(state)))
    page_content.append(Div(bold("Term:")))
    page_content.append(Div(TextBox("new_term", "")))
    page_content.append(Div(bold("Definition:")))
    page_content.append(Div(TextBox("new_definition", "")))
    page_content.append(LineBreak())
    page_content.append(Div(Button("Add Term", "save_term")))
    return Page(state, page_content)


def make_add_term_test_state() -> State:
    """Test helper: return an empty State with a pending error message, for testing add_term."""
    add_term_state = make_test_state([])
    add_term_state.message = "Error: something"
    return add_term_state


assert_equal(add_term(make_add_term_test_state()).state, make_test_state([]))


@route
def save_term(state: State, new_term: str, new_definition: str) -> Page:
    """
    Validate the Add Term form and, if it is valid, add a new Term to state.user_terms.

    The parameter names match the names of the TextBoxes on the Add Term page.
    """
    term_name = new_term.strip()
    definition = new_definition.strip()
    error_message = find_term_error(term_name, definition, state.user_terms)
    if error_message != "":
        state.message = error_message
        return add_term(state)

    state.user_terms.append(Term(term_name, definition))
    state.message = "New term added! " + term_name + TERM_SEPARATOR + definition
    return home(state)
```

```
assert_equal(save_term(make_test_state([]), "  cat ", " a small pet ").state.user_terms,
             [Term("cat", "a small pet")])
assert_equal(save_term(make_test_state([]), "", "a small pet").state.user_terms, [])
assert_equal(save_term(make_test_state(make_test_terms()), "cat",
"duplicate").state.user_terms,
             make_test_terms())


@route
def import_terms(state: State) -> Page:
    """Import Terms page: one big text area where the user pastes many 'Term - Definition'
lines."""
    page_content: list[Any] = [
        make_title("Import Terms", 1),
        Div(Button("Cancel", "home")),
        HorizontalRule()
    ]
    page_content.extend(make_message_content(take_message(state)))
    page_content.append(Div("Enter one term per line in the format: ", bold("Term - Definition")))
    # The TextArea is added directly to the page (not wrapped in a Div) so that no extra
    # whitespace gets placed inside the box. The LineBreak after it keeps the layout on separate
lines.
    page_content.append(TextArea("import_text"))
    page_content.append(LineBreak())
    page_content.append(Div(Button("Import", "save_import")))
    return Page(state, page_content)


assert_equal(import_terms(make_test_state(make_test_terms())).state,
make_test_state(make_test_terms()))


@route
def save_import(state: State, import_text: str) -> Page:
    """
    Add every line of the text area that is in the 'Term - Definition' format, then show the home
page.

    Lines that are blank, badly formatted, or duplicates of an existing term are skipped.
    The parameter name matches the name of the TextArea on the Import Terms page.
    """
    added_count, skipped_count = import_lines(import_text, state.user_terms)
```

```
    state.message = make_import_message(added_count, skipped_count)
    return home(state)


assert_equal(save_import(make_test_state([]), "cat - a small pet\nbad line\ndog - a loyal
pet").state.user_terms,
             [Term("cat", "a small pet"), Term("dog", "a loyal pet")])
assert_equal(save_import(make_test_state([]), "bad line").state.user_terms, [])


@route
def export_terms(state: State) -> Page:
    """Export Terms page: shows all terms as 'Term - Definition' lines the user can copy and
save."""
    page_content: list[Any] = [
        make_title("Export Terms", 1),
        Div(Button("Home", "home")),
        HorizontalRule()
    ]
    if len(state.user_terms) == 0:
        page_content.append(Div(italic("There are no terms to export!")))
        return Page(state, page_content)

    page_content.append(Div("Copy the text below and save it somewhere. "
                            "You can paste it into 'Import Terms' next time."))
    # PreformattedText keeps the line breaks
    page_content.append(PreformattedText(build_export_text(state.user_terms)))
    return Page(state, page_content)


assert_equal(export_terms(make_test_state(make_test_terms())).state,
make_test_state(make_test_terms()))
assert_equal(export_terms(make_test_state([])).state, make_test_state([]))


@route
def remove_term(state: State) -> Page:
    """Remove Term page: Cancel / Remove All buttons at the top, then each term with its own
Remove button."""
    page_content: list[Any] = [make_title("Remove Term", 1)]
    if len(state.user_terms) == 0:
        page_content.append(Div(Button("Home", "home")))
        page_content.append(HorizontalRule())
        page_content.append(Div(italic("You need to add a term before you can remove one!")))
```

```
        return Page(state, page_content)

    page_content.append(Div(Button("Cancel", "home"), Button("Remove All", "remove_all")))
    page_content.append(HorizontalRule())

    # Loop over the Term instances; each Remove button comes BEFORE its term.
    # Advanced technique: in make_removal_row, the dictionary {"term_index": term_index} tells
Drafter
    # to give delete_term a parameter named 'term_index' with this value when the button is
clicked.
    for term_index, term in enumerate(state.user_terms):
        page_content.append(make_removal_row(term_index, term))
    return Page(state, page_content)


assert_equal(remove_term(make_test_state(make_test_terms())).state,
make_test_state(make_test_terms()))
assert_equal(remove_term(make_test_state([])).state, make_test_state([]))


@route
def delete_term(state: State, term_index: int) -> Page:
    """Remove the Term at position `term_index` from state.user_terms, then return to the home
page."""
    if 0 <= term_index < len(state.user_terms):
        removed_term = state.user_terms[term_index].term
        state.user_terms.pop(term_index)
        state.message = removed_term + " successfully removed."
    else:
        state.message = "Error: invalid input. No term has been removed."
    return home(state)


assert_equal(delete_term(make_test_state(make_test_terms()), 1).state.user_terms,
             [Term("cat", "a small pet"), Term("fish", "lives in water"), Term("bird", "can fly")])
assert_equal(delete_term(make_test_state(make_test_terms()), 10).state.user_terms,
make_test_terms())
assert_equal(delete_term(make_test_state(make_test_terms()), -1).state.user_terms,
make_test_terms())


@route
def remove_all(state: State) -> Page:
    """Remove every Term from state.user_terms, then return to the home page."""
```

```
    removed_count = len(state.user_terms)
    state.user_terms.clear()
    state.message = "All " + str(removed_count) + " term(s) successfully removed."
    return home(state)


assert_equal(remove_all(make_test_state(make_test_terms())).state.user_terms, [])
assert_equal(remove_all(make_test_state([])).state.user_terms, [])


@route
def quiz(state: State, chosen_term: str = "") -> Page:
    """
    Show the current question (a definition and four term buttons) and process the user's answer.

    When a button is clicked, `chosen_term` holds the clicked term and is checked against the
    correct term. The default value "" lets other routes call quiz(state) just to display the question.
    """
    process_answer(state, chosen_term)
    current_term = state.quiz_terms[state.current_question]

    page_content: list[Any] = [Div(Button("Quit", "home"))]
    question_title = "Question " + str(state.current_question + 1)
    if is_retry_question(state):
        question_title = question_title + " (retry)"
    page_content.append(make_title(question_title, 2))
    page_content.append(HorizontalRule())
    page_content.append(Div(bold(current_term.definition)))
    page_content.append(LineBreak())

    if state.feedback == "":
        # one button per line
        page_content.extend(make_choice_buttons(state.answer_choices))
    else:
        page_content.append(make_feedback_component(state.feedback))
        page_content.append(LineBreak())
        page_content.append(Div(Button("Next Question", "next_question")))
    return Page(state, page_content)


assert_equal(quiz(make_quiz_state(0, make_test_terms(), "")).state, make_quiz_state(0,
make_test_terms(), ""))
assert_equal(quiz(make_quiz_state(0, make_test_terms(), ""), "cat").state.score, 1)
assert_equal(quiz(make_quiz_state(0, make_test_terms(), ""), "dog").state.score, 0)
```

```
assert_equal(quiz(make_quiz_state(0, make_test_terms(), ""), "dog").state.missed_counts,
{"cat": 1})


@route
def results(state: State) -> Page:
    """Show the final score (correct on the first try) and which terms were missed, and how many
times."""
    page_content: list[Any] = [
        make_title("Quiz Complete!", 1),
        Div(Button("Home", "home"), Button("Take Quiz Again", "start_quiz")),
        HorizontalRule()
    ]

    score_text = "Final Score: " + str(state.score) + "/" + str(len(state.user_terms))
    if state.score == len(state.user_terms):
        page_content.append(Div(bold(change_color(score_text, GOOD_COLOR))))
    else:
        page_content.append(Div(bold(score_text)))
    page_content.append(LineBreak())

    page_content.append(make_title("Missed Terms", 2))
    page_content.extend(make_missed_content(state.missed_counts))
    return Page(state, page_content)


def make_finished_quiz_state() -> State:
    """Test helper: return a State at the end of a quiz with a score of 3 and cat missed twice."""
    finished_quiz_state = make_quiz_state(4, make_test_terms(), "")
    finished_quiz_state.score = 3
    finished_quiz_state.missed_counts = {"cat": 2}
    return finished_quiz_state


assert_equal(results(make_finished_quiz_state()).state.score, 3)
assert_equal(results(make_finished_quiz_state()).state.missed_counts, {"cat": 2})


@route
def start_quiz(state: State) -> Page:
    """
    Start a new quiz: shuffle the user's terms and reset the score, question number, feedback and
missed terms.
```

```
    A quiz needs at least four terms. Because the shuffle is random, only the "not enough terms"
    case and the properties of the new quiz (not its exact order) are unit tested.
    """
    if len(state.user_terms) < MIN_TERMS_FOR_QUIZ:
        state.message = "Error: you must add at least 4 terms before you can take a quiz."
        return home(state)

    state.quiz_terms = shuffle_terms(state.user_terms)
    state.current_question = 0
    state.score = 0
    state.feedback = ""
    state.missed_counts = {}
    state.answer_choices = create_question_choices(state.quiz_terms[0], state.user_terms)
    # Advanced technique: a route can call another route function and return the Page it makes
    return quiz(state)


def first_answer_is_in_choices() -> bool:
    """Test helper: start a quiz and return True if the first question's correct term is a choice."""
    started_state = start_quiz(make_test_state(make_test_terms())).state
    return started_state.quiz_terms[0].term in started_state.answer_choices


assert_equal(start_quiz(make_test_state(make_test_terms()[:3])).state.quiz_terms, [])
assert_equal(len(start_quiz(make_test_state(make_test_terms())).state.quiz_terms), 4)
assert_equal(start_quiz(make_test_state(make_test_terms())).state.current_question, 0)
assert_equal(start_quiz(make_test_state(make_test_terms())).state.score, 0)
assert_equal(start_quiz(make_test_state(make_test_terms())).state.feedback, "")
assert_equal(start_quiz(make_test_state(make_test_terms())).state.missed_counts, {})
assert_equal(len(start_quiz(make_test_state(make_test_terms())).state.answer_choices), 4)
assert_equal(first_answer_is_in_choices(), True)


@route
def next_question(state: State) -> Page:
    """
    Move on to the next question.

    If the last answer was incorrect, that Term is appended to the end of quiz_terms so it
    has to be attempted again. If no questions remain, the results page is shown. The answer
    choices for the next question are random, so the tests only check how many there are.
    """
    if state.feedback == "":
        # The question hasn't been answered yet, so stay on it
```

```
        return quiz(state)

    if state.feedback != CORRECT_FEEDBACK:
        state.quiz_terms.append(state.quiz_terms[state.current_question])
    state.current_question += 1
    state.feedback = ""

    if state.current_question >= len(state.quiz_terms):
        return results(state)

    state.answer_choices = create_question_choices(state.quiz_terms[state.current_question],
                                                   state.user_terms)
    return quiz(state)


# unanswered question: nothing changes
assert_equal(next_question(make_quiz_state(0, make_test_terms(), "")).state,
             make_quiz_state(0, make_test_terms(), ""))
# correct answer on the last question: the quiz ends and no question is added
assert_equal(next_question(make_quiz_state(3, make_test_terms(),
CORRECT_FEEDBACK)).state.current_question, 4)
assert_equal(len(next_question(make_quiz_state(3, make_test_terms(),
CORRECT_FEEDBACK)).state.quiz_terms), 4)
# incorrect answer: the missed term is added to the end and the next question is shown
assert_equal(next_question(make_quiz_state(0, make_test_terms(), "Incorrect!
...")).state.current_question, 1)
assert_equal(len(next_question(make_quiz_state(0, make_test_terms(), "Incorrect!
...")).state.quiz_terms), 5)
assert_equal(next_question(make_quiz_state(0, make_test_terms(), "Incorrect!
...")).state.quiz_terms[4],
             Term("cat", "a small pet"))
assert_equal(next_question(make_quiz_state(0, make_test_terms(), "Incorrect!
...")).state.feedback, "")
assert_equal(len(next_question(make_quiz_state(0, make_test_terms(), "Incorrect!
...")).state.answer_choices), 4)
# incorrect answer on the last original question: the missed term is retried
assert_equal(next_question(make_quiz_state(3, make_test_terms(), "Incorrect!
...")).state.current_question, 4)
assert_equal(len(next_question(make_quiz_state(3, make_test_terms(), "Incorrect!
...")).state.quiz_terms), 5)


set_site_information(
    author="Christopher Detrick",
```

```
    description="A study tool that lets you import vocabulary terms and take a shuffled multiple-choice quiz.",
    sources=["Claude"],
    planning=[],
    links=[]
)

start_server(State([], [], 0, 0, "", [], "", {}))
```